Module V

Part I - ETHEREUM

Ethereum blockchain

Ethereum, just like any other blockchain, can be visualized as a transaction-based state machine.

The Ethereum network

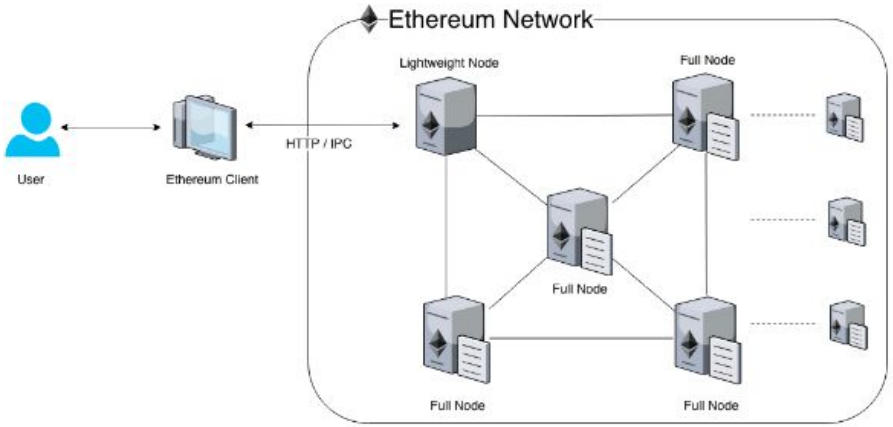
The Ethereum network is a peer-to-peer network where nodes participate in order to maintain the blockchain and contribute to the consensus mechanism.

An Ethereum network has all the nodes connected to each other using the P2P network protocol.

Each node keeps the latest copy of the Ethereum blockchain ledger.

A user can interact with the Ethereum network via the Ethereum client.

The Ethereum client can be a desktop/mobile/web page.



Ethereum high-level architecture

A full node

Maintains the complete data structure of the blockchain ledger and participates in the blockchain network by verifying every block and transaction using their consensus rules.

Full nodes always keep the latest copy of the ledger.

A light node

Does not maintain the full copy of the blockchain ledger.

It verifies the transaction using a method called Simple Payment Verification (SPV).

A light node just downloads the header information of the blocks, hence lower storage space is required in running light nodes.

Miners

Owners of the nodes willingly contribute their computing resources or hard disk space to store and validate transactions; in return, they collect block reward and transaction fees from the transactions.

The nodes that perform transaction execution and verification are called miners.

Networks can be divided into three types, based on requirements and usage. They are,

1. Mainnet

Mainnet is the current live network of Ethereum.

The current version of mainnet is Byzantium (Metropolis) and its chain ID is 1.

Chain ID is used to identify the network.

2. Testnet

Testnet is also called Ropsten and is the widely used test network for the Ethereum blockchain.

This test blockchain is used to test smart contracts and DApps before being deployed to the production live blockchain.

It allows experimentation and research.

3. Private net

This is the private network that can be created by generating a new genesis block.

This is usually the case in private blockchain distributed ledger networks, where a private group of entities start their blockchain and use it as a permissioned blockchain.

Components of the Ethereum ecosystem

- Keys and addresses
- Accounts
- Transactions
- Messages
- Ether cryptocurrency/tokens
- The EVM
- Smart contracts

1. Keys and addresses

They are used in Ethereum blockchain mainly to represent ownership and transfer of Ether.

Keys are used in pairs of private and public type.

The private key is generated randomly and is kept secret whereas a public key is derived from the private key.

Addresses are derived from the public keys which are a 20-bytes code used to identify accounts.

key generation and address derivation

First, a private key is randomly chosen (256 bits positive integer) under the rules defined by elliptic curve secp256k1 specification.

2. The public key is then derived from this private key using ECDSA (Elliptic Curve Digital Signature Algorithm) recovery function.

3. An address is derived from the public key which is the right most 160 bits of the *Keccak* (pronounced “ketchak”)hash of the public key.

Examples for keys and address (in HEX)

1. Private key:
   `b51928c22782e97cca95c490eb958b06fab7a70b9512c38c36974f47b954ffc4`
2. Public key:
   `3aa5b8eefd12bdc2d26f1ae348e5f383480877bda6f9e1a47f6a4afb35cf998ab847f1e3948b1173622dafc6b4ac198c97b18fe1d79f90c9093ab2ff9ad99260`
3. Address: `0x77b4b5699827c5c49f73bd16fd5ce3d828c36f32`

2. Accounts

Operations are performed between and on the accounts.

They result in state transitions.

The state is created or updated as a result of the interaction between accounts and transaction execution.

The state transition is achieved using the Ethereum state transition function

Types of accounts in Ethereum:

- Externally Owned Accounts (EOAs)
- Contract Accounts (CAs)

EOAs

- They are associated with a human user
- EOAs has ether balance
- They are capable of sending transactions
- They have no associated code
- They are controlled by private keys
- Accounts contain a key-value store

CAs:

- They are not associated with any user or actor on the blockchain.
- CAs have Ether balance.
- They have associated code that is kept in memory/storage on the blockchain.
- They can get triggered and execute code in response to a transaction or a message from other contracts.
- CAs contain a key-value store.

3. Transactions

A transaction in Ethereum is a digitally signed data packet using a private key that contains the instructions .

Types of transactions

On Ethereum there are a 3 different types of transactions:

Regular transactions: a transaction from one account to another.

Contract deployment transactions: a transaction without a 'to' address, where the data field is used for the contract code.

Execution of a contract: a transaction that interacts with a deployed smart contract. In this case, 'to' address is the smart contract address.

Fields in a Transaction

Nonce:

- Nonce is a number that is incremented by one every time a transaction is sent by the sender.
- It must be equal to the number of transactions sent and is used as a unique identifier for the transaction.
- A nonce value can only be used once.
- This is used for replay protection on the network

Gas price:

- Represents the amount of Wei required to execute the transaction. (Wei is the smallest denomination of ether.)
- This is charged for all computation costs incurred as a result of the execution of this transaction.

Gas limit:

The maximum amount of gas that can be consumed to execute the transaction.

To (RECIPIENT):

The address of the recipient of the transaction. This is a 20-byte value.

Value:

The total number of Wei to be transferred to the recipient.

Signature:

- Signature is composed of three fields, namely V, R, and S.
- R and S represent digital signature information and V is used to recover public key.

Init:

- The code contained in this field is executed only once when the account is created for the first time, it (init) gets destroyed immediately after that.
- Used only in contract creation transactions.

Data:

- If the transaction is a message call, then the data field is used instead of init.
- It represents the input data of the message call.

```
{
  to: "0x2c8645BFE28BEEb6E19843eE9573b7539DD5B530", // Bob
  gasLimit: "21000",
  maxFeePerGas: "30", // 28 (base) + 2 (priorityFee)
  maxPriorityFeePerGas: "2", // minerTip
  nonce: "0",
  value: "100000000000000000", // 1 ether worth of wei
  data: '0x', // no data, we are not interacting with a contract
  type: 2, // this is not a legacy tx
  chainId: 4, // this is AU, we deal only in test networks! (Göerli)
}
```

Transaction object example

4. Messages

A message is a data packet passed between two accounts.

This data packet contains data and value (amount of ether).

It can either be sent via a smart contract (autonomous object) or from an external actor (externally owned account) in the form of a transaction that has been digitally signed by the sender.

Contracts can send messages to other contracts.

Messages only exist in the execution environment and are never stored.

A message consists of the following components

- The sender of the message
- Recipient of the message
- Amount of Wei to transfer and message to the contract address
- Optional data field (Input data for the contract)
- The maximum amount of gas that can be consumed

5. Ether cryptocurrency / tokens (ETC and ETH)

As an incentive to the miners, Ethereum also rewards its own native currency called Ether, abbreviated as ETH.

There are now two Ethereum blockchains:

one is called Ethereum Classic (original version of Ethereum), and its currency is represented by ETC, whereas the hardforked version is ETH (continues to grow).

Ether is minted by miners as a currency reward for the computational effort they spend to secure the network by verifying and with validation transactions and blocks.

Ether is used within the Ethereum blockchain to pay for the execution of contracts on the EVM.

Ether is used to purchase gas as crypto fuel, which is required to perform computation on the Ethereum blockchain.

Unit	Alternative name	Wei value	Number of Weis
Wei	Wei	1 Wei	1
KWei	Babbage	1^3 Wei	1,000
MWei	Lovelace	1^6 Wei	1,000,000
Gwei	Shannon	1^9 Wei	1,000,000,000
Micro Ether	Szabo	1^12 Wei	1,000,000,000,000
Milli Ether	Finney	1^15 Wei	1,000,000,000,000,000
Ether	Ether	1^18 Wei	1,000,000,000,000,000,000

Ether Denominations

6. The Ethereum Virtual Machine (EVM)

EVM is a simple stack-based execution machine.

It runs bytecode instructions to transform the system state from one state to another.

The word size of the virtual machine is set to 256-bit.

The stack size is limited to 1024 elements and is based on the Last In, First Out (LIFO) queue.

EVM is a Turing-complete machine.

gas fee is required to run any instruction.

This means that infinite loops that can result in denial of service attacks are not possible due to gas requirements.

EVM also supports exception handling.

In case exceptions occur, such as not having enough gas or invalid instructions, in which case the machine would immediately halt and return the error to the executing agent.

EVM is an entirely isolated and sandboxed runtime environment.

The code that runs on the EVM does not have access to any external resources, such as a network or filesystem.

This results in increased security, deterministic execution and allows untrusted code (anyone can run code) to be run on Ethereum blockchain.

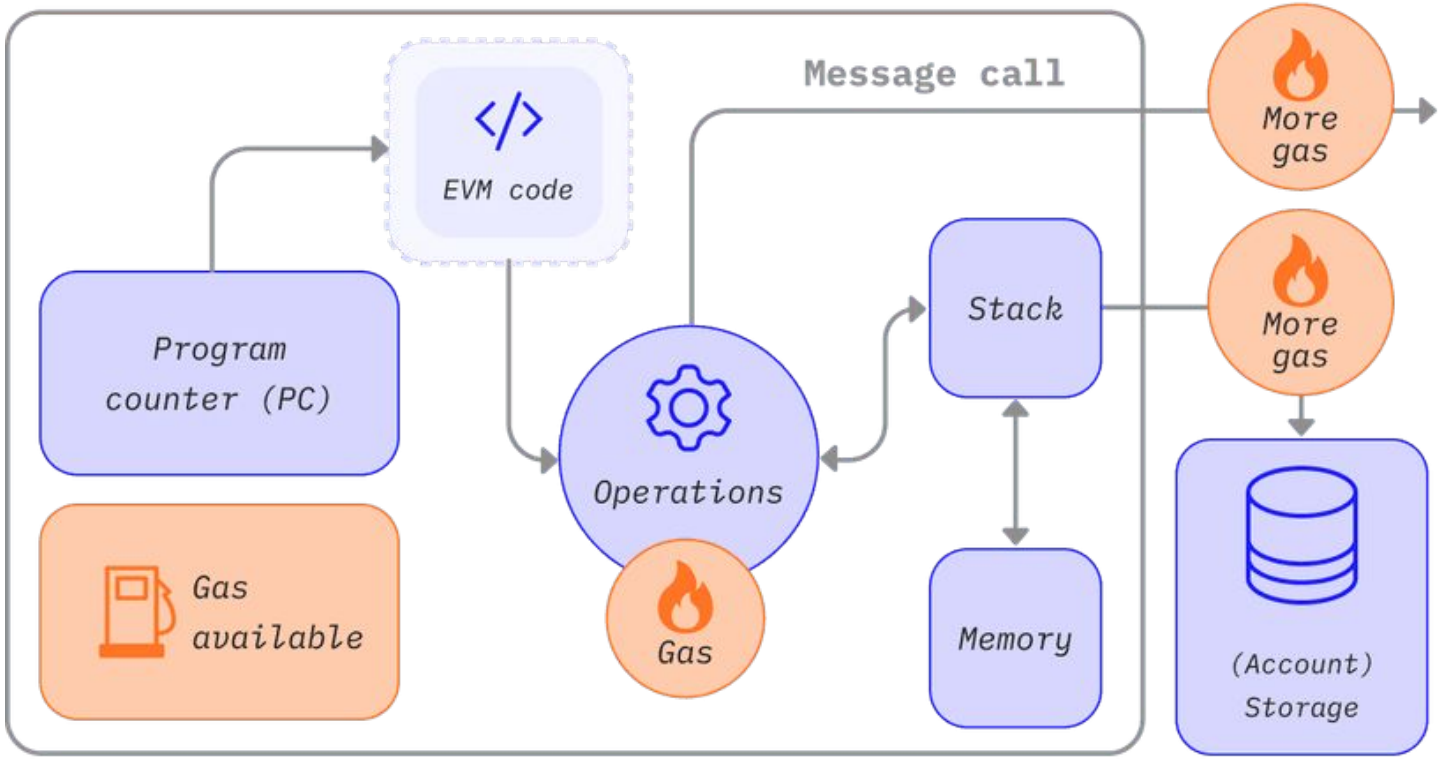
There are two types of storage available to contracts and EVM.

Memory:

- ➔ It is a byte array.
- ➔ When a contract finishes the code execution, the memory is cleared.
- ➔ It is similar to RAM.

Storage

- ➔ Permanently stored on the blockchain.
- ➔ It is a key value store and can be thought of like a hard disk storage



Ethereum Virtual machine

WebAssembly (WASM) aims to be able to run machine code in the browser that will result in execution at native speed.

WASM is developed by Google, Mozilla, and Microsoft and is now being designed as an open standard by the W3C community group.

Similarly, the aim of EVM 2.1 is to be able to run the EVM instruction set (opcodes) natively in CPUs, thus making it faster and efficient.

A language called Joyfully Universal Language for (Inline) Assembly (JULIA) can compile to various backends such as EVM and eWASM.

7. Smart contracts

There are various contracts that are available in the precompiled format in Ethereum blockchain to support different functions.

These contracts, known as precompiled contracts or native contracts

They are available natively on the blockchain to support various computationally intensive tasks.

They run on the local node and are coded within the Ethereum client.

In addition, users can write user defined smart contracts using solidity language.

Native contracts

There are eight precompiled contracts.

- ➔ The SHA-256-bit hash function
- ➔ The RIPEMD-160-bit hash function
- ➔ The elliptic curve public key recovery function
- ➔ The identity/datacopy function
- ➔ Elliptic Curve (EC) point addition function
- ➔ Elliptic Curve (EC) scalar multiplication:
- ➔ Elliptic Curve (EC) pairing
- ➔ Big mod exponentiation function:

Gas

Gas is the fuel that allows Ethereum to operate, in the same way that a car needs gasoline to run.

Gas refers to the unit that measures the amount of computational effort required to execute specific operations on the Ethereum network.

Each Ethereum transaction requires computational resources to execute.

Payment for computation is made in the form of a gas fee.

Gas fee = Amount of gas required to do some operation * Cost per unit gas (Gas Price).

The fee is paid regardless of whether a transaction succeeds or fails.

Every bytecode operation in EVM has fixed gas units assigned. When that operation is executed by EVM, fixed quantity of gas is consumed by the transaction.

Gas fees have to be paid in Ethereum's native currency, ether (ETH).

Gas prices are usually quoted in gwei

Gas Fee Calculation

You can set the amount of gas you are willing to pay when you submit a transaction.

By offering a certain amount of gas, you are bidding for your transaction to be included in the next block.

If you offer too little, validators are less likely to choose your transaction for inclusion, meaning your transaction may execute late or not at all.

The total gas fee is divided into two components: the base fee and the priority fee (tip).

$$Total\ Gas\ Fee = units\ of\ gas\ used * (base\ fee + priority\ fee)$$

For example,

Jordan has to pay Taylor 1 ETH. An ETH transfer requires 21,000 units of gas, and the base fee is 10 gwei. Jordan includes a tip of 2 gwei.

The total fee would now be equal to: units of gas used * (base fee + priority fee)

where the base fee is a value set by the protocol and the priority fee is a value set by the user as a tip to the validator.

I.e. Total gas fee = 21,000 * (10 + 2) = 252,000 gwei (0.000252 ETH).

When Jordan sends the money, 1.000252 ETH will be deducted from Jordan's account.

Taylor will be credited 1.0000 ETH.

The validator receives the tip of 0.000042 ETH.

The base fee of 0.00021 ETH is burned

Gas limit

The gas limit refers to the maximum amount of gas you are willing to consume on a transaction.

A standard ETH transfer requires a gas limit of 21,000 units of gas.

For example, if you put a gas limit of 50,000 for a simple ETH transfer, the EVM would consume 21,000, and you would get back the remaining 29,000.

However, if you specify too little gas, for example, a gas limit of 20,000 for a simple ETH transfer, the EVM will consume your 20,000 gas units attempting to fulfill the transaction, but it will not complete.

The EVM then reverts any changes, but since the validator has already done 20k gas units worth of work, that gas is consumed.

Gas price

The gas price is the price per gas unit you have to pay for executing your transaction.

The unit of gas price is always defined in gwei.

$$Gas\ Price = base\ fee + priority\ fee$$

'Base Fee' is constant and defined by EVM, *To increase "Gas Price",* We have to increase *'Tip(priority fee)'*

According to your gas price, miners decide to process your transaction.

If the transaction has a higher gas price, there is a chance that your transaction will be processed early.

If your gas price is low, then your transaction might be processed after a delay or when the blockchain is free to process low gas price transactions.

Formulas

The following are some formulas to calculate the gas *usage* and *transaction fee* per transaction:

$$TxFee_{Max} = Gas_{Limit} * GasPrice$$
$$TxFee_{Paid} = Gas_{Used} * GasPrice$$
$$TxFee_{Returned} = TxFee_{Max} - TxFee_{Paid}$$
$$TxFee_{Returned} = Gas_{Unused} * GasPrice$$
$$Gas_{Unused} = Gas_{Limit} - Gas_{Used}$$
$$BalanceRequiredAtTransactionInitiation = EtherValueToTransfer + TxFee_{Max}$$

Example

Let's take an example of a transaction where you want to transfer 0.1 ETH to your friend. The following are the values for the transaction fields you would have to fill while creating a transaction:

Field name	Value
From	0xff899af34214b0d777bcd3c230ef637b763f0b01
To	0xc4fe5518f0168da7bbafe375cd84d30f64cda491
Value	0.1 ether
Gas limit	30,000
Gas price	0.000000021 ether (21 gwei)

Let's calculate the values using these formulas: (Assume one transfer transaction requires 21,000 gas units)

TxFeeMax = 30,000 * 0.000000021 = 0.00063 ether

TxFeePaid = 21,000 * 0.000000021 = 0.000441 ether

TxFeeReturned = 0.00063 - 0.000441 = 0.000189 ether

GasUnused = 30,000 - 21,000 = 9,000 units of gas

BalanceRequiredAtTransactionInitiation = 0.1 + 0.00063 = 0.10063 ether

Block difficulty in Ethereum

Block difficulty level controls the creationof new blocks in Ethereum.

Block difficulty is increased if the time between two blocks decreases, whereas it increases if the block time between two blocks decreases.

This is required to maintain a roughly consistent block generation time.

The difficulty adjustment algorithm in Ethereum's Homestead release is shown as follows:

if the time difference between the generation of the parent block and the current block is less than 10 seconds, the difficulty goes up.

If the time difference is between 10 to 19 seconds, the difficulty level remains the same.

Finally, if the time difference is 20 seconds or more, the difficulty level decreases.

This decrease is proportional to the time difference.

In addition to timestamp-difference-based difficulty adjustment, there is also another part that increases the difficulty exponentially after every 100,000 blocks.

This is the so-called Difficulty Time Bomb or Ice Age introduced in the Ethereum network, which will make it very hard to mine on the Ethereum blockchain at some point in the future.

$$d_c = d_p + \underbrace{\frac{d_p}{2048} * \max\left(1 - \left\lfloor \frac{ts_c - ts_p}{10} \right\rfloor, -99\right)} + \underbrace{2^{\left(\left\lfloor \frac{n_p + 1}{100{,}000} \right\rfloor - 2\right)}}$$

The current difficulty value d_c is the previous value d_p plus two components. The first component adjusts the difficulty level based on the number of seconds between the current block and the previous block. With $s = ts_c - ts_p$ and $s < 10$, the factor, $\max(x)$, evaluates to 1 (assuming integer division). In this case, the block time is too fast and the difficulty increases thereby slowing down block production. If $10 \leq s < 20$, the factor $\max(x)$ becomes 0 which insures the difficulty does not adjust when s is near the target of 14 seconds. If $s \geq 20$, the value of the factor begins a steady decrease to -99 which results in lowering the difficulty thereby increasing block production. The second component of the equation is based on the block number of the previous block. The integer division by 100,000 in the exponent creates a step function with a period of 100,000 blocks. This second component is what is commonly referred to as the difficulty bomb. Without the second component, there would be no difficulty bomb.

Block Difficulty Calculation

Ethereum Blockchain and Blocks

The blockchain maintains a series of blocks.

Each block is linked together with the last generated block.

A block contains multiple transactions.

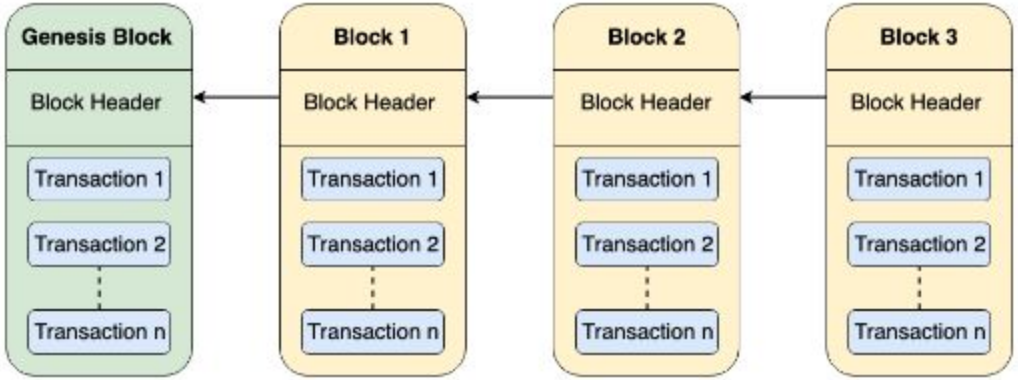
Ethereum uses PoS (Proof of Stake).

In Proof of Stake, the energy requirement to become a validator is much lower and can be done by individuals without a high overhead energy cost.

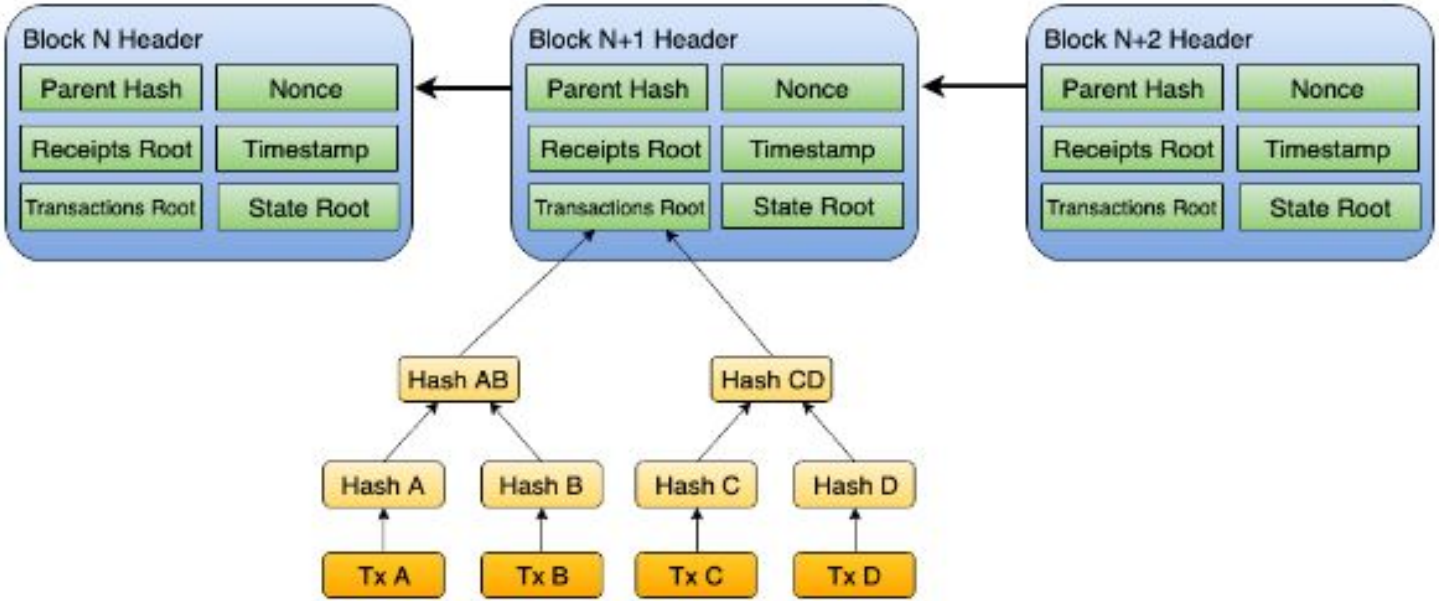
This encourages more users to become validators, decreasing the centralization risk, and thereby increasing the security of the network.

Validators are required to stake 32ETH by depositing it into a contract to have the ability to validate blocks.

If any given validator acts dishonestly or maliciously, they put themselves at risk of losing their staked ETH.



Ethereum Blockchain



Ethereum Block Header and Transaction Root

The Parent Hash always points to the parent block of the newly generated block. This child-to-parent link goes on until the genesis block.

The Nonce is the number that is found by the miner to solve the cryptographic puzzle.

The Receipts Root is the Keccak 256-bit hash of the root node of the tree structure, which is populated with the receipts of each transaction.

The Timestamp is the time when the block is found and added to the blockchain.

The Transactions Root is the Keccak 256-bit hash of the root node of the tree structure, which is populated with each transaction.

The State Root is the Keccak 256-bit hash of the root node of the state tree, after all transactions are executed.